

Securing Network Services and Ports

1. List Open Ports.

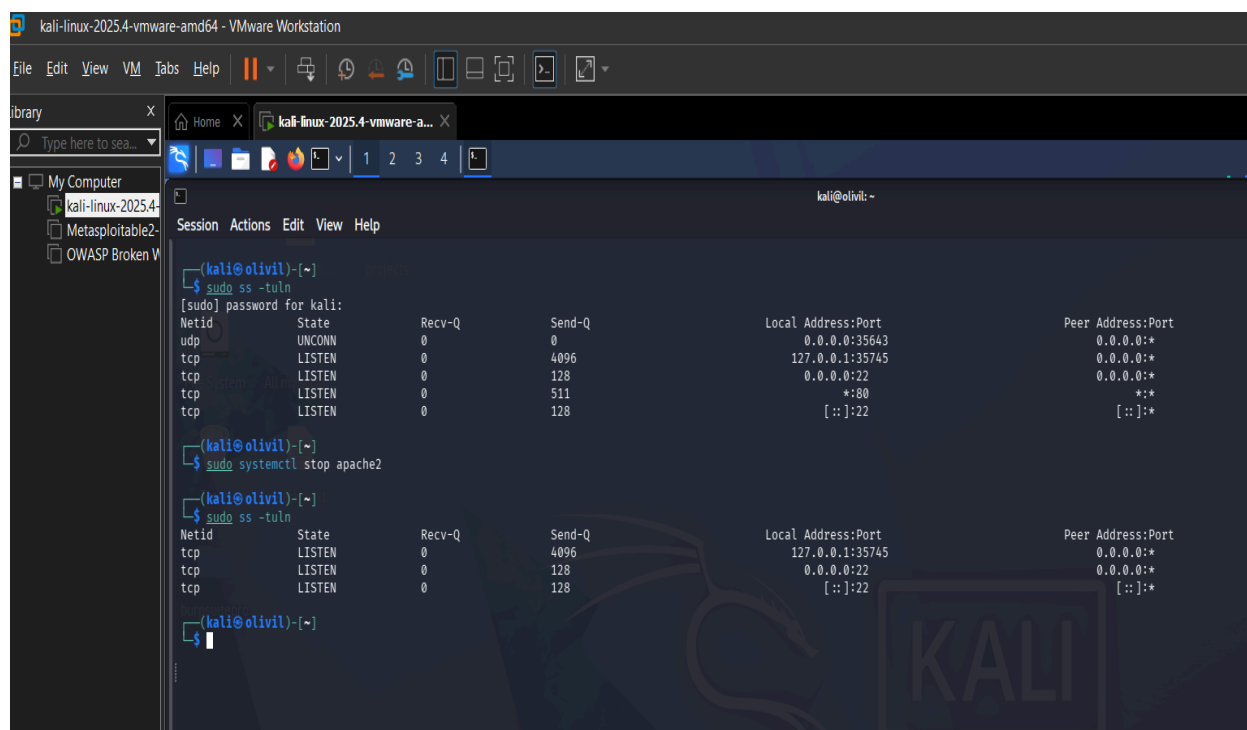
Command Used:

```
sudo ss -tln
```

2. Closed Unnecessary Ports by:

- Stopping an unnecessary service Apache:

```
sudo systemctl stop apache2 sudo systemctl disable apache2
```



- The ss command successfully displayed the active listening ports on the Kali system. Ports 22 (SSH) and 80 (HTTP/Apache) were identified as open TCP ports, along with local UDP/TCP services.
- Apache was successfully stopped and disabled, and port 80 was no longer listening. This confirmed that the unnecessary network service was successfully closed.

Exercise 4: SSH and Remote Access

1. Secured the SSH Service by:
 - Editing the SSH configuration file.

Commands Used:

`sudo nano /etc/ssh/sshd_config`

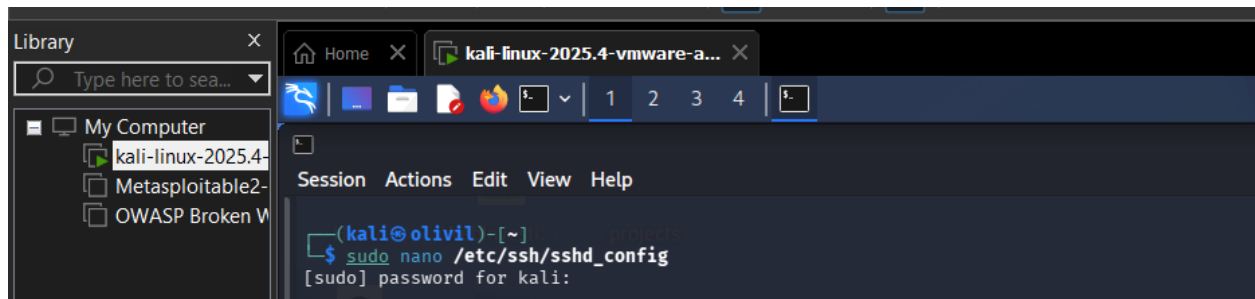
- Changed the SSH port #Port22 to 2222

Then saved and exit:

- Ctrl + O

- Press Enter
- Ctrl + X

2. Restarted SSH using:
`sudo systemctl restart sshd`



The image shows a Kali Linux virtual machine running on VMware. The terminal window displays the contents of the `/etc/ssh/sshd_config` file, which is being edited with the `nano` text editor. The file contains various configuration options for the SSH daemon, including listening port, host keys, ciphers, logging, and authentication settings. The `Port 2222` line is highlighted. The interface includes a menu bar, a toolbar, and a sidebar with a file manager view.

```
File Edit View VM Tabs Help
kali-linux-2025.4-vmware-a...
Type here to sea...
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W

Session Actions Edit View Help
GNU nano 9.1 /etc/ssh/sshd_config *

# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH

#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

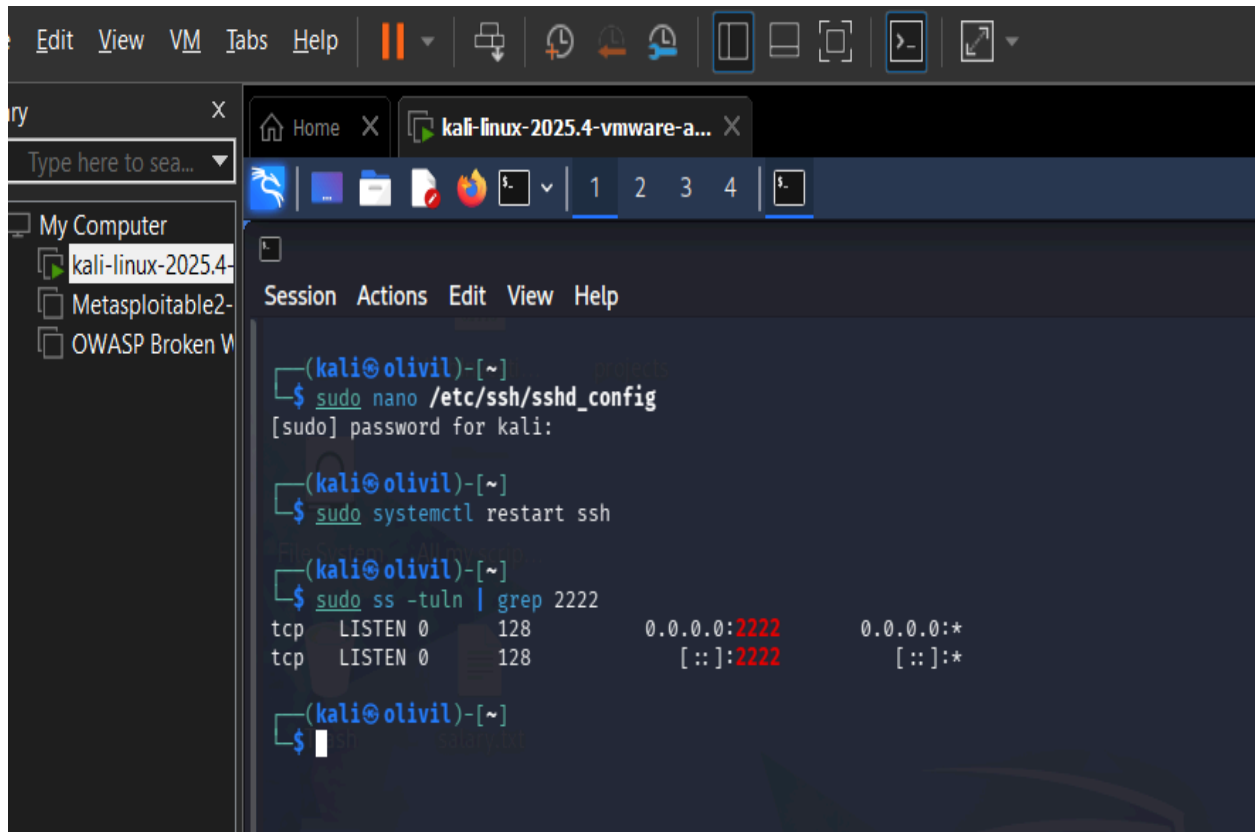
#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_/ Go To Line  M-E
```

to direct input to this VM, move the mouse pointer inside or press Ctrl+G.



```
(kali@olivil)-[~]
$ sudo nano /etc/ssh/sshd_config
[sudo] password for kali:

(kali@olivil)-[~]
$ sudo systemctl restart ssh

(kali@olivil)-[~]
$ sudo ss -tuln | grep 2222
tcp    LISTEN 0      128          0.0.0.0:2222      0.0.0.0:*
tcp    LISTEN 0      128          [::]:2222       [::]:*
```

SSH was successfully reconfigured to use port 2222 instead of port 22. The `ss` output confirmed that SSH is listening on port 2222 for both IPv4 and IPv6 connections.

Exercise 5: System Update and Patching

1. Check for Updates.

Commands used:

```
sudo apt update
```

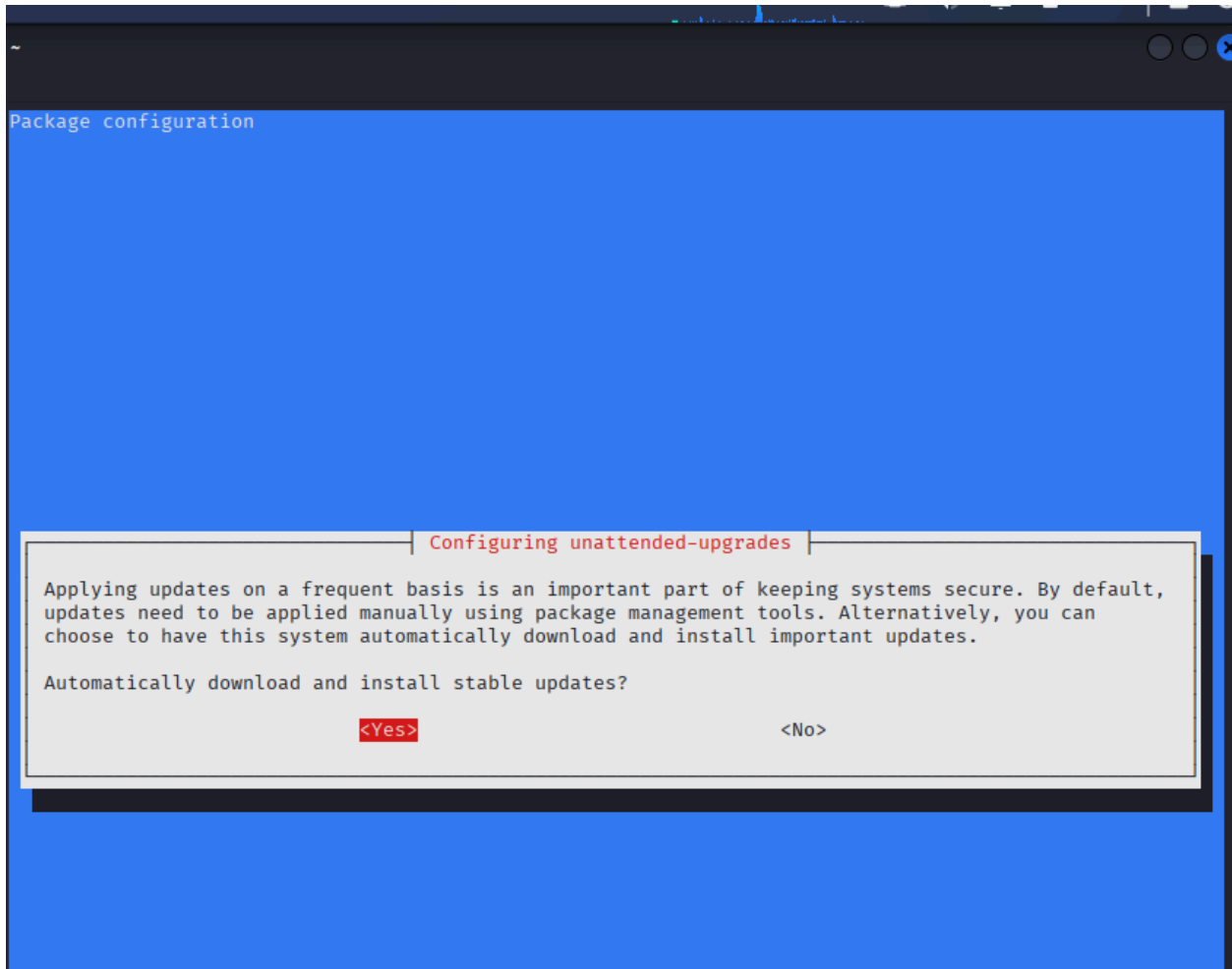
```
sudo apt upgrade
```

2. Automate Updates:

- Install unattended-upgrades:

```
sudo apt install unattended-upgrades
```

```
sudo dpkg-reconfigure unattended-upgrades
```

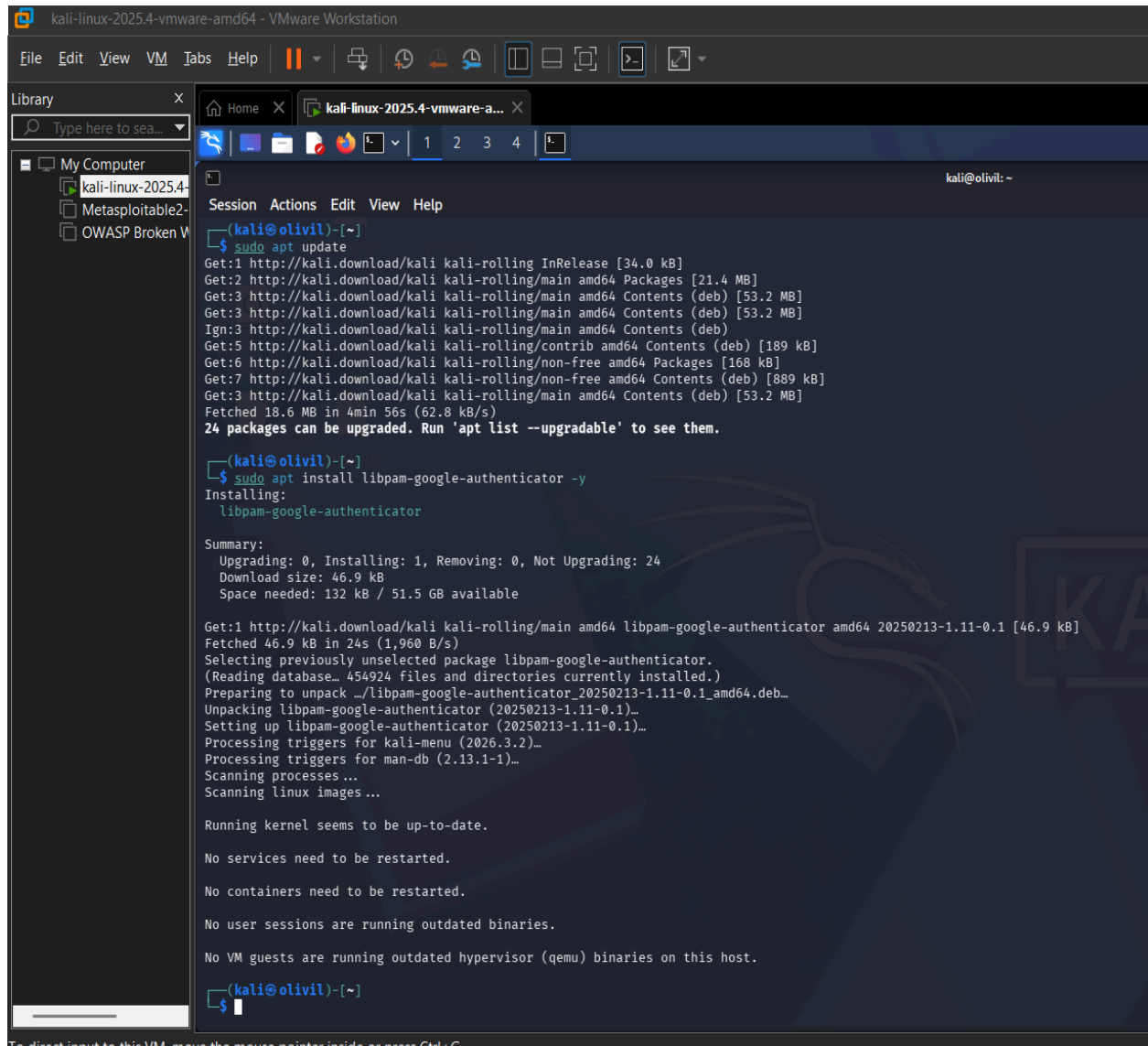



The system was successfully updated using `apt update` and `apt upgrade`. The `unattended-upgrades` package was installed and automatic stable updates were enabled to help keep the system secure.

Implementing Two-Factor Authentication (2FA) with Google Authenticator

Step 1: Installed Google Authenticator

1. Update the system package index:
`sudo apt update`
2. Install the Google Authenticator PAM module:
`sudo apt install libpam-google-authenticator -y`



```
kali@olivil:~$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.4 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.2 MB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.2 MB]
Ign:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb)
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [189 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [168 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [889 kB]
Get:8 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.2 MB]
Fetched 18.6 MB in 4min 56s (62.8 kB/s)
24 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@olivil)-[~]
$ sudo apt install libpam-google-authenticator -y
Installing:
  libpam-google-authenticator

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 24
  Download size: 46.9 kB
  Space needed: 132 kB / 51.5 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 libpam-google-authenticator amd64 20250213-1.11-0.1 [46.9 kB]
Fetched 46.9 kB in 24s (1,960 B/s)
Selecting previously unselected package libpam-google-authenticator.
(Reading database... 454924 files and directories currently installed.)
Preparing to unpack .../libpam-google-authenticator_20250213-1.11-0.1_amd64.deb...
Unpacking libpam-google-authenticator (20250213-1.11-0.1)...
Setting up libpam-google-authenticator (20250213-1.11-0.1)...
Processing triggers for kali-menu (2026.3.2)...
Processing triggers for man-db (2.13.1-1)...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.

(kali@olivil)-[~]
$ dpkg-query -f='${Package} ${Version} ${Architecture}\n'
```

The Google Authenticator PAM module was successfully installed after updating the system package index. The system is now ready for the user-specific 2FA configuration.

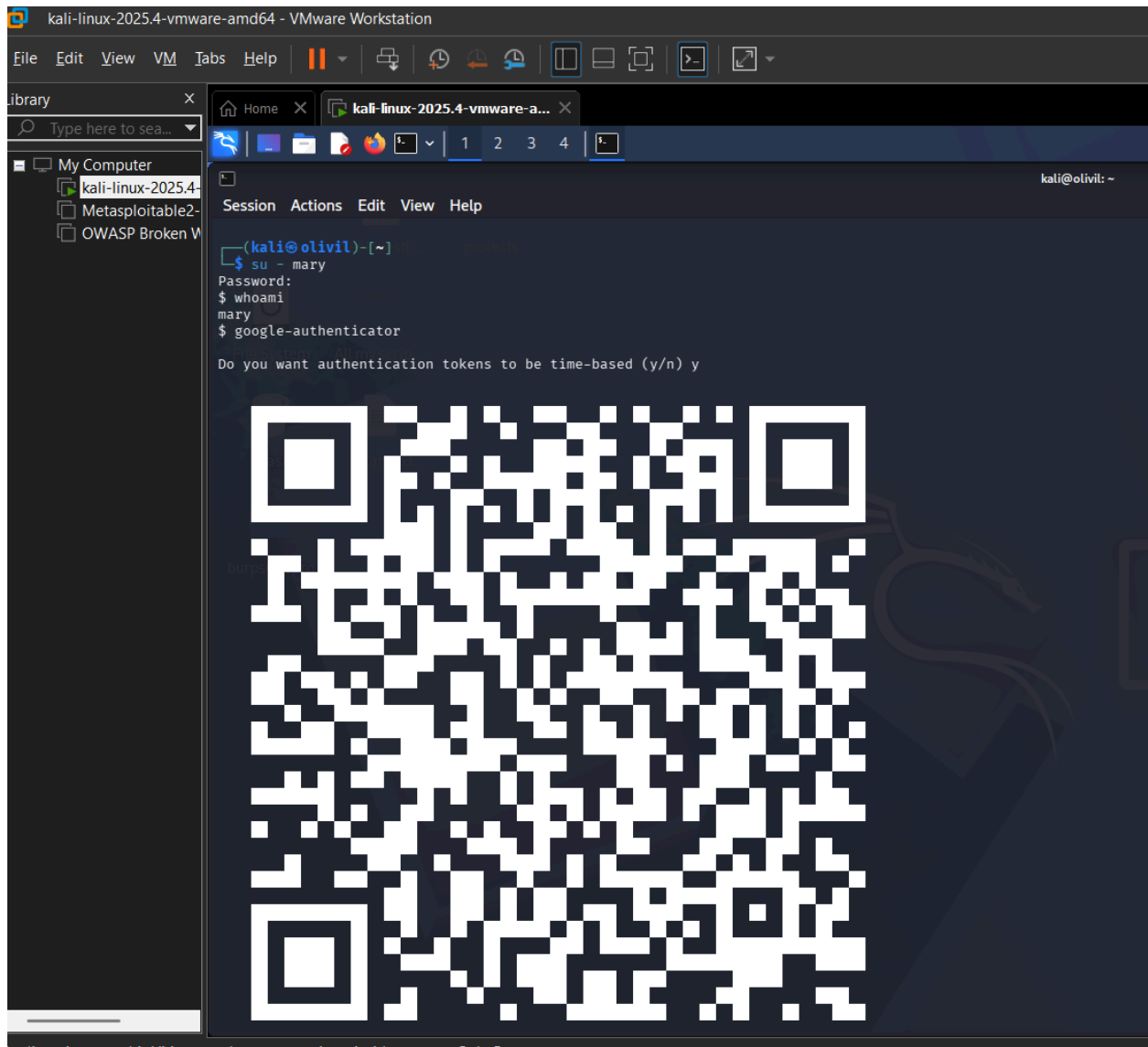
Step 2: Configure Google Authenticator for Your User

1. Switch to the user account you want to configure 2FA for:
su - <username>

2. Run the Google Authenticator setup:
google-authenticator

3. Follow the prompts:

- QR Code: Scan the QR code using the Google Authenticator app on your smartphone.
- Emergency Codes: Save the emergency backup codes provided (these can be used if you lose access to the app).
- Enable Time-Based Tokens: Answer yes when asked if you want to enable time-based tokens.



```

Your new secret key is: GQHMA5MU4UVUQKGZI23STPIHH2A3IZHA
Enter code from app (-1 to skip): 335465
Code incorrect (correct code 234305). Try again.
Enter code from app (-1 to skip): 800636
Code incorrect (correct code 917509). Try again.
Enter code from app (-1 to skip): 925904
Code confirmed
Your emergency scratch codes are:
11235064
30937035
83024280
66932568
52036224

Do you want me to update your "/home/mary/.google_authenticator" file? (y/n) y

Do you want to disallow multiple uses of the same authentication
token? This restricts you to one login about every 30s, but it increases
your chances to notice or even prevent man-in-the-middle attacks (y/n) y

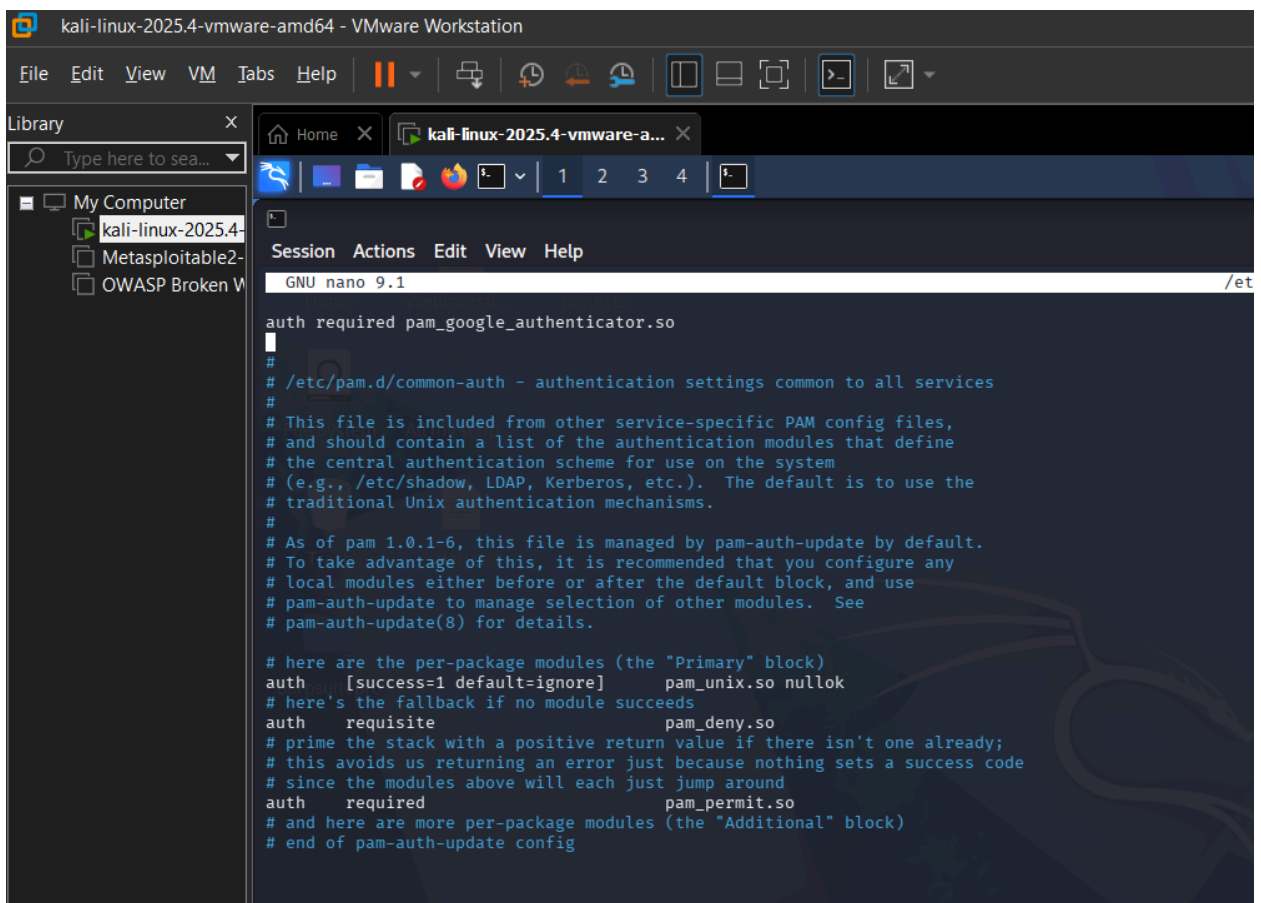
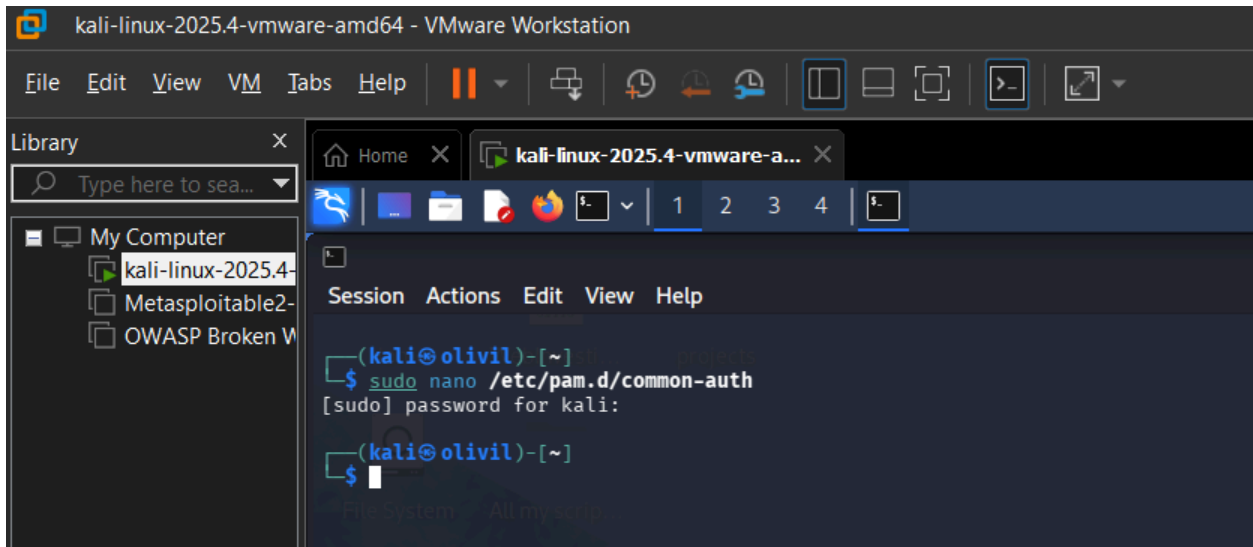
By default, a new token is generated every 30 seconds by the mobile app.
In order to compensate for possible time-skew between the client and the server,
we allow an extra token before and after the current time. This allows for a
time skew of up to 30 seconds between authentication server and client. If you
experience problems with poor time synchronization, you can increase the window
from its default size of 3 permitted codes (one previous code, the current
code, the next code) to 17 permitted codes (the 8 previous codes, the current
code, and the 8 next codes). This will permit for a time skew of up to 4 minutes
between client and server.
Do you want to do so? (y/n) n

If the computer that you are logging into isn't hardened against brute-force
login attempts, you can enable rate-limiting for the authentication module.
By default, this limits attackers to no more than 3 login attempts every 30s.
Do you want to enable rate-limiting? (y/n) y
$ exit
(kali@olivil)-[~]
```

Google Authenticator was successfully configured for the **Mary** account, including time-based authentication, token reuse protection, and rate limiting. The emergency backup codes were also generated successfully.

Step 3: Configure PAM (Pluggable Authentication Modules) for 2FA

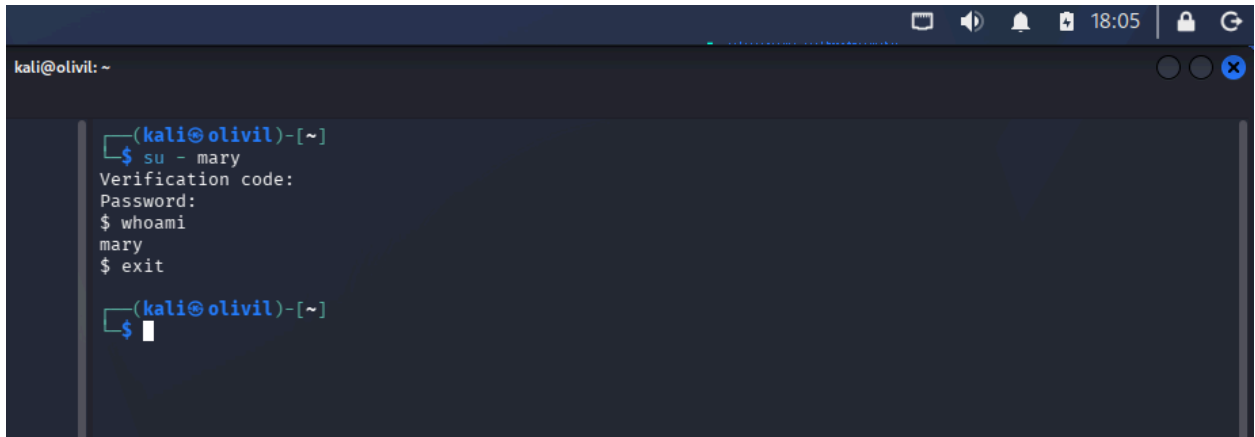
1. Open the common-auth file:
`sudo nano /etc/pam.d/common-auth`
2. Add the following line at the top of the file:
`auth required pam_google_authenticator.so`
3. Save and exit the file (Ctrl+O, Enter, Ctrl+X).



The PAM configuration was successfully updated to include Google Authenticator, enabling 2FA authentication for the system.

Step 4: Test 2FA Configuration

1. Open a new terminal or SSH session and attempt to log in as the configured user.
2. After entering the username and password, you should be prompted for the 2FA token generated by the Google Authenticator app.

A terminal window titled 'kali@olivil: ~' with standard Linux window controls. The terminal shows a user switching to 'mary' using 'su - mary'. They are prompted for a 'Verification code:' and a 'Password:'. After entering these, they run 'whoami' which outputs 'mary', and then 'exit' to return to the root prompt. The prompt then changes back to '(kali@olivil)-[~]' and a new shell prompt '\$' appears.

```
(kali@olivil)-[~]  
$ su - mary  
Verification code:  
Password:  
$ whoami  
mary  
$ exit  
  
(kali@olivil)-[~]  
$
```

Mary successfully logged in by providing both her Google Authenticator verification code and password, confirming that the 2FA setup is working correctly

Implementing Key-Based Authentication and Securing SSH Access

Scenario

You are a cybersecurity analyst at SecureTech Ltd. In addition to disabling root login, the company wants to implement key-based authentication for enhanced SSH security. Your Exercise is to configure SSH for key-based authentication, test the setup, and ensure the root user is securely disabled.

Part 1: SSH Key-Based Authentication

Commands used:

1. Verified the current user and account details

```
whoami  
id  
echo $USER
```

2. Generated a 4096-bit RSA SSH key pair

```
ssh-keygen -t rsa -b 4096
```

This created:

```
/home/kali/.ssh/id_rsa  
/home/kali/.ssh/id_rsa.pub
```

3. Checked the SSH service

```
sudo systemctl status ssh
```

```

kali@olivil: ~
$ whoami
kali

kali@olivil: ~
$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),300(hark),130(kaboxer)

kali@olivil: ~
$ echo $USER
kali

kali@olivil: ~
$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kali/.ssh/id_rsa):
Enter passphrase for "/home/kali/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_rsa
Your public key has been saved in /home/kali/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:MLLvoMqH/wGLIFJf/iGClenk6z2Y+q0U1HwJZFzerw kali@olivil
The key's randomart image is:
+--[RSA 4096]--+
| .o+o. |
| .. ++. |
| +++=o |
| oo=0+ooo |
| *B +.. .S |
| 0.o o.E |
| .=..... |
| =o.= o. |
| =*Boo.. |
+--[SHA256]--+

kali@olivil: ~
$ sudo systemctl status ssh
[sudo] password for kali:
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-08-15 06:55:56 EDT; 32min ago
   Invocation: c9528cdf1bb840f5b449a2f94dcab702
   Docs: man:sshd(8)
         man:sshd_config(5)
   Process: 997 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1009 (sshd)
```

The kali user was verified successfully, and an RSA 4096-bit SSH key pair was generated. The private key was stored in `/home/kali/.ssh/id_rsa`, while the public key was stored in `/home/kali/.ssh/id_rsa.pub`. The SSH service was checked and confirmed to be active and running, with the SSH server listening on port 22.

Part 2: Copy Public Key and Test SSH Login

Commands used

1. Copied the public key to the SSH server

ssh-copy-id kali@localhost

2. Accepted the server's host key

yes

3. Authenticate to install the key

kali@localhost's password:

The key was successfully installed, confirmed by:

Number of key(s) added: 1

4. Tested SSH key-based authentication

ssh kali@localhost

5. Exited the SSH session

exit

```
kali@olivil: ~  
  
(kali@olivil)-[~]  
$ ssh-copy-id kali@localhost  
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kali/.ssh/id_rsa.pub"  
The authenticity of host 'localhost (::1)' can't be established.  
ED25519 key fingerprint is: SHA256:6L5RA5iA+bVS5/4jJDR1KbKK4Uznkb47CV3Bw+lZU4  
This host key is known by the following other names/addresses:  
  ~/.ssh/known_hosts:1: [hashed name]  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed  
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys  
kali@localhost's password:  
Permission denied, please try again.  
kali@localhost's password:  
  
Number of key(s) added: 1  
  
Now try logging into the machine, with: "ssh 'kali@localhost'"  
and check to make sure that only the key(s) you wanted were added.  
  
(kali@olivil)-[~]  
$ ssh kali@localhost  
Linux olivil 7.0.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.0.12-2kali1 (2026-06-18) x86_64  
  
The programs included with the Kali GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
You have mail.  
Last login: Fri Jul 10 05:09:58 2026 from ::1  
(kali@olivil)-[~]  
$ exit  
Connection to localhost closed.  
  
(kali@olivil)-[~]  
$
```

The SSH public key was successfully copied to the kali account on the local SSH server. Although the first password attempt was unsuccessful, the key installation completed successfully, with one key added. The SSH connection was then tested successfully using `ssh kali@localhost`, confirming that the configured SSH key could authenticate the kali user. The SSH session was closed normally using `exit`.

Part 3: Configure SSH for Key-Based Authentication

Commands used:

1. Edited the SSH configuration file

```
sudo nano /etc/ssh/sshd_config
```

2. Configured public-key authentication

```
PubkeyAuthentication yes
```

3. Disabled password authentication

```
PasswordAuthentication no
```

Then:

CTR + O = SAVE

CTR + X =SAVE

4. Verified the SSH configuration

```
sudo sshd -t
```

5. Restarted the SSH service

```
sudo systemctl restart ssh
```

6. Verified that SSH is running

```
sudo systemctl status ssh --no-pager
```

7. Tested key-based login

```
ssh kali@localhost
```

8. Verified the logged-in user

```
whoami
```

```
(kali@olivil)-[~]  
$ sudo nano /etc/ssh/sshd_config  
[sudo] password for kali:  
(kali@olivil)-[~]
```

```
#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key
```

```
# Ciphers and keying
#RekeyLimit default none
```

```
# Logging
#SyslogFacility AUTH
```

```
#LogLevel INFO
```

```
# Authentication:
```

```
#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10
```

```
PubkeyAuthentication yes
```

```
# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2
```

```
#AuthorizedPrincipalsFile none
```

```
#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody
```

```
# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes
```

```
# To disable tunneled clear text passwords, change to "no" here!
```

```
PasswordAuthentication yes
#PermitEmptyPasswords no
```

```
^G Help
^X Exit
```

```
^O Write Out
^R Read File
```

```
^F Where Is
^_ Replace
```

```
^K Cut
^U Paste
```

```
^T Execute
^J Justify
```

```
^C Location
^_ Go To Line
```

```
M-U Undo
M-E Redo
```

```
M-A Set Mark
M-6 Copy
```

```
M-J
^B
```

```

(kali@olivil)-[~]
$ sudo sshd -t

(kali@olivil)-[~]
$ sudo systemctl restart ssh

(kali@olivil)-[~]
$ sudo systemctl status ssh --no-pager
ssh.service - OpenBSD Secure Shell server
  Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
  Active: active (running) since Sat 2026-08-15 07:55:41 EDT; 1min 35s ago
  Invocation: d4f137b13de749179203c493cea672b6
  Docs: man:sshd(8)
       man:sshd_config(5)
  Process: 27433 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 27436 (sshd)
  Tasks: 1 (limit: 2066)
  Memory: 1.3M (peak: 2.3M)
  CPU: 57ms
  CGroup: /system.slice/ssh.service
          └─27436 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 15 07:55:41 olivil systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Aug 15 07:55:41 olivil sshd[27436]: Server listening on 0.0.0.0 port 22.
Aug 15 07:55:41 olivil sshd[27436]: Server listening on :: port 22.
Aug 15 07:55:41 olivil systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

(kali@olivil)-[~]
$ ssh kali@localhost
linux olivil 7.0.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.0.12-2kali1 (2026-06-18) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
You have mail.
Last login: Sat Aug 15 07:37:32 2026 from ::1
(kali@olivil)-[~]
$ whoami

```

The SSH server was configured to allow public-key authentication and disable password-based SSH authentication. The configuration was successfully validated using `sshd -t` with no errors. The SSH service restarted successfully and remained active and running. A new SSH session was then established successfully using the configured SSH key, and `whoami` confirmed that the authenticated user was `kali`.

Part 2: Disable Root Login

Commands used:

```
sudo nano /etc/ssh/sshd_config
```

Changed:

```
PermitRootLogin no
```

Then:

```
CRL + O = SAVE
```

```
CRL + X = EXIT
```

Configuration tested with:

```
sudo sshd -t
```

SSH restarted with:

```
sudo systemctl restart ssh
```

SSH service verified with:

```
sudo systemctl status ssh --no-pager
```

Normal key-based login tested with:

```
ssh kali@localhost
```

```
whoami
```

Root login tested with:

```
ssh root@localhost
```

```
kali-linux-2025.4-vmware-a...
kali@olivil:
Session Actions Edit View Help
(kali@olivil)-[~]
$ sudo nano /etc/ssh/sshd_config
[sudo] password for kali:
(kali@olivil)-[~]
$ sudo sshd -t
(kali@olivil)-[~]
$ sudo systemctl restart ssh
(kali@olivil)-[~]
$ sudo systemctl status ssh --no-pager
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-08-15 08:41:17 EDT; 17s ago
     Invocation: 0c6f71a8147847dc91aa953708c886fa
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 50380 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 50391 (sshd)
      Tasks: 1 (limit: 2066)
     Memory: 1.4M (peak: 2.8M)
        CPU: 53ms
    CGroup: /system.slice/ssh.service
            └─50391 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 15 08:41:17 olivil systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Aug 15 08:41:17 olivil sshd[50391]: Server listening on 0.0.0.0 port 22.
Aug 15 08:41:17 olivil systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Aug 15 08:41:17 olivil sshd[50391]: Server listening on :: port 22.
(kali@olivil)-[~]
$ sudo sshd -t
```

```
File Edit View VM Tabs Help
kali-linux-2025.4-vmware-a...
Type here to sea...
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken V...

Session Actions Edit View Help
kali@oliv

(kali@olivil)-[~]
$ ssh kali@localhost
Linux olivil 7.0.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.0.12-2kali1 (2026-06-18) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
You have mail.
Last login: Sat Aug 15 07:58:35 2026 from ::1
(kali@olivil)-[~]
$ whoami
kali

(kali@olivil)-[~]
$ exit
Connection to localhost closed.

(kali@olivil)-[~]
$ ssh root@localhost
root@localhost's password:
Permission denied, please try again.
root@localhost's password: 
```

SSH was configured to prevent direct root login by setting PermitRootLogin no. The configuration passed the SSH syntax check, and the SSH service restarted successfully. Key-based authentication for the kali user continued to work normally. A direct SSH attempt as root returned Permission denied, confirming that root SSH access was successfully disabled.
